

BEGINNER → ADVANCED • ENGLISH + मराठी • INTERVIEW READY

Shell Scripting Notes

शेल स्क्रिप्टिंग नोट्स — सखोल आणि सोप्या मराठीत

A deep, beginner-to-advanced guide to Linux/Bash shell scripting — with detailed explanations, "how it works" notes, common mistakes, and interview Q&A for every topic, in both English and Marathi.

लिनक्स/Bash शेल स्क्रिप्टिंगचे सखोल मार्गदर्शन — प्रत्येक विषयासाठी सविस्तर स्पष्टीकरण, सामान्य चुका, आणि इंटरव्यू प्रश्नोत्तरांसह, इंग्रजी आणि मराठी दोन्ही भाषांत.

Variables

Loops

Functions

Arrays

sed / awk

Debugging

Interview Q&A

Common Mistakes

Shell Scripting — Deep Notes

इंग्रजी + मराठी सखोल स्पष्टीकरण, सामान्य चुका आणि इंटरव्यू प्रश्नोत्तरांसह

BEGINNER / नवशिक्या

1 What is a Shell Script?

शेल स्क्रिप्ट म्हणजे काय ?

EN A shell is a program that sits between you and the Linux kernel — it reads the commands you type and asks the kernel to carry them out. A shell SCRIPT is simply a text file full of these commands, written in order, so instead of typing 10 commands one by one every day, you write them once and run the file whenever you want. This saves time and avoids mistakes from typing the same thing repeatedly. Bash (Bourne Again SHell) is the most common shell used for scripting on Linux.

MR शेल हा एक प्रोग्राम आहे जो तुमच्या आणि Linux कर्नलच्या मध्ये काम करतो — तुम्ही टाईप केलेली कमांड तो वाचतो आणि कर्नलला ती पार पाडायला सांगतो. शेल स्क्रिप्ट म्हणजे अशाच कमांड्सची क्रमवार यादी असलेली एक टेक्स्ट फाईल. रोज १० कमांड्स एकेक टाईप करण्याऐवजी त्या एकदाच लिहून फाईल हवं तेव्हा रन करता येते. यामुळे वेळ वाचतो आणि वारंवार टाईप करताना होणाऱ्या चुका टळतात. Linux वर स्क्रिप्टिंगसाठी सगळ्यात जास्त वापरला जाणारा शेल म्हणजे Bash.

HOW IT WORKS Kernel = the core of the OS that manages hardware. Shell = the interface (like bash, sh, zsh). Script = a saved list of shell commands.

Example / उदाहरण

```
# my first comment - iti is a comment line
echo "Hello, this is my first shell script"
```

Save this as hello.sh

⚠ Common Mistakes / सामान्य चुका

- Thinking 'shell' and 'terminal' are the same thing — terminal is just the window; shell is the program running inside it.
- Forgetting that a script is just a plain text file — it needs no special 'compiling'.

Interview Questions & Answers

Q: What is the difference between a shell and a terminal?

A: Terminal is the window/application you open. Shell is the program (like bash) running inside that window which actually interprets your commands.

Q: Name a few common shells.

A: sh, bash, zsh, ksh, csh — bash is the most widely used for scripting on Linux.

Q: Is a shell script compiled or interpreted?

A: Interpreted — each line is read and executed one at a time by bash, there is no separate compile step.

शेबॅंग (Shebang) आणि स्क्रिप्ट रन करणे

EN The very first line of a script, `#!/bin/bash`, is called the SHEBANG. Even though it looks like a comment (starts with `#`), the kernel specifically reads this line to know which program should interpret the rest of the file. Without it, the script may run using the wrong shell and behave unexpectedly. After writing a script you must give it 'execute' permission using `chmod` before Linux will let you run it directly.

MR स्क्रिप्टच्या पहिल्या ओळीला, `#!/bin/bash`, 'शेबॅंग' म्हणतात. ती कॉमेंटसारखी दिसत असली (`#` ने सुरू होते) तरी कर्नल ही ओळी खास वाचतो, जेणेकरून बाकीची फाईल कोणत्या प्रोग्रामने चालवायची हे कळते. ती नसेल तर स्क्रिप्ट चुकीच्या शेलने चालू शकते आणि अनपेक्षित वागू शकते. स्क्रिप्ट लिहिल्यावर ती थेट चालवण्यासाठी आधी `chmod` वापरून तिला 'execute' परवानगी द्यावी लागते.

HOW IT WORKS `chmod +x file.sh` sets the 'x' (execute) bit on the file. `./file.sh` tells the shell 'run the executable file located in this folder'. Running it as 'bash file.sh' ignores the shebang and always uses bash directly — useful even without execute permission.

Example / उदाहरण

```
#!/bin/bash
echo "Script chalu zhali!"

# Terminal madhe:
# chmod +x hello.sh      -> execute permission denyesathi
# ./hello.sh             -> script run karnyasathi
# bash hello.sh          -> ha pan ek prayaI marg aahe
```

⚠ Common Mistakes / सामान्य चुका

- Putting a space after `#!` (must be exactly `#!/bin/bash`, no space).
- Forgetting `chmod +x` and then wondering why `./script.sh` gives 'Permission denied'.
- Using `#!/bin/sh` but writing bash-only syntax like arrays — `sh` is more limited than `bash`.

Interview Questions & Answers

Q: What happens if you forget the shebang line?

A: The script may still run if you call it with 'bash script.sh', but running './script.sh' directly could use the wrong default shell and cause syntax errors.

Q: How do you make a script executable?

A: `chmod +x script.sh` — this adds the execute permission for the file.

Q: Difference between `./script.sh` and `bash script.sh`?

A: `./script.sh` uses the shebang line to decide the interpreter and requires execute permission. `bash script.sh` explicitly runs it with bash regardless of the shebang or permissions.

3 Variables

व्हेरिएबल्स (Variables)

EN A variable is a named container that stores a value (text or number) so you can reuse it later instead of typing it again and again. In bash, you do NOT declare a data type — everything is treated as a string by default, even numbers, unless you explicitly do math with it. Writing the value uses just name=value with absolutely no spaces around the '='. To READ the value back, you put a dollar sign in front: \$name.

MR व्हेरिएबल म्हणजे एक नाव दिलेली जागा जिथे एखादी किंमत (मजकूर किंवा नंबर) साठवली जाते, जेणेकरून ती वारंवार टाईप न करता पुन्हा वापरता येईल. Bash मध्ये डेटा टाईप सांगायला लागत नाही — नंबर असला तरी डिफॉल्टने तो टेक्स्ट (string) म्हणूनच धरला जातो, जोपर्यंत तुम्ही त्यावर गणित करत नाही. किंमत ठेवण्यासाठी name=value असे लिहायचे, '=' च्या आजूबाजूला अजिबात space नको. किंमत परत वाचण्यासाठी नावासमोर डॉलर चिन्ह लावायचे: \$name.

HOW IT WORKS Bash stores variables in memory as key-value pairs for the current shell session. 'local' variables exist only inside a function; 'export'ed variables become 'environment variables' visible to child processes too.

Example / उदाहरण

```
#!/bin/bash
name="Rahul"
age=21

echo "Namaskar, $name!"
echo "Tumche vay $age varshe aahe."

# Chukiche : name = "Rahul"    (spaces mule error yeto)

# Environment variable banvanyasathi:
export CITY="Pune"
```

⚠ Common Mistakes / सामान्य चुका

- Adding spaces around '=' (name = "Rahul") — bash treats this as a command, not assignment.
- Forgetting the \$ when reading a value (echo name instead of echo \$name).
- Not quoting variables that may contain spaces, causing word-splitting bugs: echo \$name vs echo "\$name".

Interview Questions & Answers

Q: Do you need to declare a data type for a bash variable?

A: No. Bash is untyped by default — every variable is stored as a string, though it can be used in arithmetic contexts.

Q: What is the difference between a local shell variable and an environment variable?

A: A local variable exists only in the current shell. Using 'export' makes it an environment variable, which is passed down to any child process/script started from that shell.

Q: Why does 'name = value' fail in bash?

A: Because of the spaces, bash tries to run a command called 'name' with arguments '=' and 'value', instead of doing an assignment.

Q: How do you make a variable read-only?

A: Using 'readonly name=value' — after this, trying to reassign it will throw an error.

कॉमेंट्स (Comments)

EN A comment is text in your script that bash completely ignores when running — it exists purely to help a human (often future-you) understand what the code does. In bash, anything after a '#' symbol, up to the end of that line, is a comment. Good scripts use comments to explain WHY something is done, not just WHAT — since the code itself already shows what.

MR कॉमेंट म्हणजे स्क्रिप्टमधला असा मजकूर जो bash रन करताना पूर्णपणे दुर्लक्षित करतो — तो फक्त माणसाला (बऱ्याचदा भविष्यातल्या तुम्हालाच) कोड समजण्यासाठी असतो. Bash मध्ये '#' नंतर त्या ओळीच्या शेवटपर्यंत लिहिलेली प्रत्येक गोष्ट कॉमेंट असते. चांगल्या स्क्रिप्टमध्ये कॉमेंट्स 'काय' केले आहे हे नाही तर 'का' केले आहे हे सांगतात — कारण कोड आधीच 'काय' दाखवतो.

Example / उदाहरण

```
#!/bin/bash
# hi script user che nav vicharte
echo "Tumche nav kay aahe?" # ha inline comment aahe

# Bash madhe multi-line comment nasato,
# pratyek olila # lavave lagte.
```

⚠ Common Mistakes / सामान्य चुका

- Believing bash has a multi-line /* */ comment like C — it does not; every line needs its own #.
- Over-commenting obvious code, or under-commenting tricky logic.

Interview Questions & Answers

Q: Does bash support multi-line comments?

A: Not natively like /* */ in C. Every commented line needs its own #. (A common trick is using a ': <<COMMENT ... COMMENT' block, but it's a workaround, not a real feature.)

Q: Are comments removed before execution or just skipped?

A: They are simply skipped — bash reads the line, sees #, and ignores everything after it on that line; nothing is 'compiled out' since there's no compile step.

युजरकडून इनपुट घेणे (read)

EN The 'read' command makes a script interactive — it pauses execution and waits for the user to type something on the keyboard and press Enter, then stores whatever was typed into a variable. This is how scripts ask questions instead of only working with fixed values. You can also add a prompt message directly using `read -p` so you don't need a separate `echo` line, and `read -s` to hide input (useful for passwords).

MR 'read' कमांडमुळे स्क्रिप्ट इंटरॲक्टिव्ह बनते — ती थांबते आणि युजर कीबोर्डवर काही टाईप करून Enter दाबेपर्यंत वाट पाहते, नंतर टाईप केलेली गोष्ट एका व्हेरिएबलमध्ये साठवते. यामुळेच स्क्रिप्ट फक्त ठराविक किंमतींऐवजी प्रश्न विचारून काम करू शकते. `read -p` वापरून वेगळी `echo` ओळ न लिहिता थेट प्रश्न दाखवता येतो, आणि पासवर्डसारख्या गोष्टींसाठी `read -s` वापरून इनपुट लपवता येते.

Example / उदाहरण

```
#!/bin/bash
echo "Tumche nav taka:"
read username
echo "Swagat aahe, $username!"

# ek ollit sudha lihu shakto:
read -p "Tumche vay taka: " age
echo "Tumhi $age varshanche aahat."

# password lapavnyasathi:
read -sp "Password taka: " pass
echo
echo "Password milale (dakhavale jat nahi)"
```

⚠ Common Mistakes / सामान्य चुका

- Forgetting that 'read' without a variable name stores input in the special variable `REPLY`.
- Not quoting the variable later, causing issues if input has spaces.

Interview Questions & Answers

Q: How do you read multiple values into separate variables in one line?

A: `read var1 var2 var3` — typing 'a b c' and pressing Enter puts a in var1, b in var2, and c (plus any leftover words) in var3.

Q: How do you hide user input while typing, like a password field?

A: Use `read -s` (silent mode) — it reads input without echoing it back to the screen.

Q: What variable holds the input if you just run 'read' with no variable name?

A: The built-in variable `REPLY`.

कमांड लाईन आर्ग्युमेंट्स

EN Command line arguments are values you pass to a script at the moment you run it, right after its name — for example `./script.sh Mumbai Pune`. Inside the script, bash automatically gives you special variables to access them: `$1` is the first argument, `$2` the second, and so on, `$0` is the script's own name, `$#` is the total count of arguments given, and `$@` gives you all arguments as a list. This lets one script behave differently depending on what's passed to it, without editing the code.

MR कमांड लाईन आर्ग्युमेंट्स म्हणजे स्क्रिप्ट रन करतानाच, तिच्या नावानंतर दिलेल्या किंमती — उदा. `./script.sh Mumbai Pune`. स्क्रिप्टमध्ये त्या मिळवण्यासाठी bash आपोआप खास व्हेरिएबल्स देतो: `$1` म्हणजे पहिला आर्ग्युमेंट, `$2` दुसरा, वगैरे. `$0` म्हणजे स्क्रिप्टचेच नाव, `$#` म्हणजे दिलेल्या आर्ग्युमेंट्सची एकूण संख्या, आणि `$@` म्हणजे सर्व आर्ग्युमेंट्सची यादी. यामुळे कोड न बदलता एकच स्क्रिप्ट वेगवेगळ्या इनपुटप्रमाणे वेगळे काम करू शकते.

Example / उदाहरण

```
#!/bin/bash
echo "Script che nav: $0"
echo "Pahila argument: $1"
echo "Dusra argument: $2"
echo "Sagle arguments: $@"
echo "Ekun arguments: $#"
```

```
if [ $# -eq 0 ]; then
    echo "Tumhi kahich argument dile nahi!"
fi
```

```
# Run karnyasathi:
# ./script.sh Mumbai Pune
```

⚠ Common Mistakes / सामान्य चुका

- Confusing `$@` and `$*` — they look similar but behave differently when quoted ("`$@`" keeps each argument separate, "`$*`" joins them into one string).
- Not checking `$#` before using `$1`, `$2` — leads to empty/unset variable errors if fewer arguments were given.

Interview Questions & Answers

Q: What is the difference between "`$@`" and "`$*`"?

A: "`$@`" expands each argument as a separate quoted word (safer for loops with spaced values). "`$*`" joins all arguments into a single string separated by the first character of IFS (space by default).

Q: How do you check how many arguments were passed to a script?

A: Use the special variable `$#`.

Q: What does `$0` represent?

A: The name (or path) used to invoke the script itself, not an argument.

INTERMEDIATE / मध्यम स्तर

अंकगणित ऑपरेटर्स (Arithmetic Operators)

EN Since bash treats everything as text by default, you need a special syntax to do math: `$(( expression ))` is the modern, preferred way — it directly evaluates a mathematical expression and returns the result. The older `'expr'` command also works but is slower and needs spaces between every operator. Common operators are `+` `-` `*` `/` (integer division only, no decimals) and `%` which gives the remainder after division.

MR Bash डिफॉल्टने सगळं टेक्स्ट म्हणून धरत असल्यामुळे गणित करण्यासाठी खास पद्धत लागते: `$(( expression ))` ही आधुनिक आणि जास्त वापरली जाणारी पद्धत आहे — ती थेट गणितीय एक्सप्रेशन काढते आणि उत्तर देते. जुनी `'expr'` कमांडही चालते पण ती स्लो आहे आणि प्रत्येक ऑपरेटरच्या आजूबाजूला space लागते. सामान्य ऑपरेटर्स आहेत `+` `-` `*` `/` (फक्त पूर्णांक भागाकार, दशांश नाही) आणि `%` जो भागाकारानंतर उरलेली बाकी देतो.

HOW IT WORKS Bash arithmetic only works with whole numbers (integers) — `7/2` gives 3, not 3.5. For decimal math you need an external tool like `'bc'` (e.g. `echo "7/2" | bc -l`).

Example / उदाहरण

```
#!/bin/bash
a=10
b=3

echo "Berij: $((a + b))"
echo "Vajabaki: $((a - b))"
echo "Guna: $((a * b))"
echo "Bhagakar: $((a / b))"          # 3 (dashansh nahi!)
echo "Baki (modulus): $((a % b))"

# dashansh (decimal) hava asel tar:
echo "Dashansh bhagakar: $(echo "$a / $b" | bc -l)"

# expr वापरून (juni padhni, space asne garjeche):
expr $a + $b
```

△ Common Mistakes / सामान्य चुका

- Forgetting that bash integer division truncates decimals — `7/2` gives 3, not 3.5.
- Using `expr` without spaces around operators (`expr 5+3` fails; `expr 5 + 3` works).
- Trying arithmetic directly on strings without `$(( ))`, which just concatenates text instead.

Interview Questions & Answers

Q: Does bash support floating-point (decimal) arithmetic natively?

A: No, bash's built-in `$(( ))` only handles integers. For decimals you must use an external tool like `'bc'` or `'awk'`.

Q: What does `10 % 3` give in bash?

A: 1 — the remainder after dividing 10 by 3.

Q: What are two ways to do arithmetic in bash?

A: `$(( expression ))` (modern, preferred) and the `'expr'` command (older, needs spaces around operators).

Conditional Statements (if-else)

कंडिशनल स्टेटमेंट्स (if-else)

EN An 'if' statement lets your script make decisions — it checks whether a condition is true, and only runs a block of code when it is. The condition is written inside square brackets [condition], and bash provides special comparison operators for this: -eq (equal), -ne (not equal), -gt (greater than), -lt (less than), -ge, -le for numbers. For comparing text/strings, you use = and != instead. 'elif' lets you check additional conditions, and 'else' is the fallback if none matched.

MR 'if' स्टेटमेंटमुळे स्क्रिप्टला निर्णय घेता येतो — तो एखादी अट (condition) खरी आहे का ते तपासतो, आणि खरी असेल तरच पुढचा कोड चालवतो. अट square brackets मध्ये [condition] लिहितात, आणि नंबर तुलना करण्यासाठी bash कडे खास ऑपरेटर्स आहेत: -eq (बरोबर), -ne (बरोबर नाही), -gt (मोठा), -lt (लहान), -ge, -le. टेक्स्ट/स्ट्रिंग तुलनेसाठी = आणि != वापरतात. जास्त अटी तपासण्यासाठी 'elif' आणि कोणतीच अट खरी नसेल तर शेवटी 'else' वापरतात.

HOW IT WORKS [] is actually a command (same as 'test'). Modern bash also supports [[]] which is more powerful (supports pattern matching, && and || directly inside).

Example / उदाहरण

```
#!/bin/bash
read -p "Number taka: " num

if [ $num -gt 0 ]; then
    echo "$num he positive number aahe"
elif [ $num -lt 0 ]; then
    echo "$num he negative number aahe"
else
    echo "Number shunya (zero) aahe"
fi

# -eq (barabar) -ne (na barabar) -gt (motha) -lt (lahan)
# -ge (motha kinva barabar) -le (lahan kinva barabar)

# string tulnesathi = ani != vapar#
name="admin"
if [ "$name" = "admin" ]; then
    echo "Tumhi admin aahat"
fi
```

⚠ Common Mistakes / सामान्य चुका

- Using == for numbers (that's for strings) or -eq for strings — mixing these up gives errors.
- Forgetting the spaces inside brackets: [\$num -gt 0] fails, [\$num -gt 0] works.
- Not quoting variables in string comparisons, causing 'unary operator expected' errors when the variable is empty.

Interview Questions & Answers

Q: What is the difference between [] and [[]] in bash?

A: [] is the older POSIX 'test' command with stricter syntax rules. [[]] is a bash-specific keyword that is more forgiving, supports pattern matching, and lets you use && / || directly inside the brackets.

Q: How do you compare two strings in bash?

A: Using = or == inside [] or [[]], e.g. ["\$a" = "\$b"]. Numbers use -eq, -ne, -gt, -lt instead.

Q: Why might [\$var -eq 5] fail with an error if \$var is empty?

A: Because the empty variable disappears entirely, leaving [-eq 5] which is invalid syntax — always quote variables like ["\$var" -eq 5] to avoid this.

केस स्टेटमेंट (case)

EN The 'case' statement is bash's version of a switch-statement found in other languages. Instead of writing many if-elif-elif blocks to compare one variable against many possible values, 'case' lets you list each possible value (called a pattern) followed by ;; to mark the end of that block. It also supports wildcard patterns like * to catch anything that didn't match earlier options, making menu-driven scripts much cleaner to read.

MR 'case' स्टेटमेंट म्हणजे इतर भाषांमधल्या switch-statement सारखा प्रकार. एका व्हेरिएबलची अनेक शक्य किंमतींशी तुलना करण्यासाठी अनेक if-elif लिहिण्याऐवजी, 'case' मध्ये प्रत्येक शक्य किंमत (pattern) लिहून शेवटी ;; लावतात, जे त्या भागाचा शेवट दाखवते. * सारखे wildcard pattern वापरून आधीच्या कोणत्याच पर्यायाशी न जुळणारी गोष्ट पकडता येते, त्यामुळे मेनू-आधारित स्क्रिप्ट्स जास्त वाचनीय बनतात.

Example / उदाहरण

```
#!/bin/bash
read -p "Fal (apple/mango/other): " fruit

case $fruit in
    apple)
        echo "Tumhi Safarchand nivadle" ;;
    mango|banana)
        echo "Tumhi Amba kinva Kela nivadla" ;;
    *)
        echo "He fal olakhat nahi" ;;
esac

# Menu style udaharan:
read -p "1)Start 2)Stop 3)Exit -> " choice
case $choice in
    1) echo "Service Start hot aahe..." ;;
    2) echo "Service Stop hot aahe..." ;;
    3) exit 0 ;;
    *) echo "Chukicha pryaay" ;;
esac
```

⚠ Common Mistakes / सामान्य चुका

- Forgetting the double semicolon ;; at the end of each pattern block.
- Forgetting the closing 'esac' (case spelled backwards).
- Not using * as a default/catch-all pattern, so unexpected input silently does nothing.

Interview Questions & Answers

Q: When would you prefer 'case' over multiple 'if-elif' statements?

A: When comparing one variable against many fixed values — case is more readable, and supports pattern matching (like *, ?, and | for multiple values in one pattern) that if-elif can't do as cleanly.

Q: What does the | mean inside a case pattern?

A: It works as OR — mango|banana) matches if the value is either 'mango' or 'banana'.

Q: What keyword ends a case block?

A: esac (case spelled backwards) — bash uses this pattern for other blocks too (if...fi, do...done).

10 for Loop

for लूप

EN A 'for' loop repeats a block of commands once for every item in a list — the list can be a set of words, a range of numbers, or even the output of another command like filenames matching a pattern. It is the right choice when you already know (or can generate) the exact set of items to repeat over, unlike a while loop which is based on a condition being true or false.

MR 'for' लूप एखाद्या यादीतील प्रत्येक घटकासाठी कमांड्सचा भाग एकदा-एकदा चालवतो — ही यादी शब्दांची, नंबरच्या रेंजची, किंवा एखाद्या पॅटर्नशी जुळणाऱ्या फाईलनावांसारख्या दुसऱ्या कमांडच्या आउटपुटचीही असू शकते. ज्या वस्तूवर लूप फिरवायचा आहे त्याची यादी आधीच माहीत असेल (किंवा तयार करता येत असेल) तेव्हा 'for' वापरणे योग्य असते — while लूप मात्र अट खरी/खोटी यावर आधारित असतो.

Example / उदाहरण

```
#!/bin/bash
for i in 1 2 3 4 5
do
    echo "Aakada: $i"
done

# Range वापरून (C-style पान चालते)
for ((i=1; i<=5; i++))
do
    echo "C-style loop: $i"
done

for i in {1..5}
do
    echo "Range varun: $i"
done

# File च्या list वर
for file in *.txt
do
    echo "Sapadli file: $file"
done
```

⚠ Common Mistakes / सामान्य चुका

- Forgetting 'do' and 'done' keywords — bash needs both to know where the loop body starts and ends.
- Not quoting file/variable names inside the loop (for file in *.txt; echo \$file — breaks with filenames containing spaces).

Interview Questions & Answers

Q: What are the two main styles of writing a for loop in bash?

A: The list-style: for item in list; do ... done, and the C-style: for ((i=0; i<n; i++)); do ... done.

Q: How would you loop through numbers 1 to 100?

A: for i in {1..100}; do ... done — or the C-style for ((i=1; i<=100; i++)).

Q: How does a for loop differ from a while loop in terms of use case?

A: for is used when the set of items to iterate is known/generatable in advance; while is used when looping should continue based on a condition, and the number of iterations may not be known beforehand.

11 while Loop

while लूप

EN A 'while' loop keeps repeating its block of commands for as long as its condition remains true — it checks the condition BEFORE every iteration, and stops the moment the condition becomes false. This is the natural choice when you don't know in advance exactly how many times you need to repeat, for example reading a file line-by-line until there are no more lines, or repeating until the user enters a valid choice.

MR 'while' लूप त्याची अट (condition) खरी राहतेपर्यंत कमांड्स पुन्हा पुन्हा चालवतो — प्रत्येक फेरीच्या आधी तो अट तपासतो, आणि अट खोटी झाली की लगेच थांबतो. किती वेळा रिपीट करायचे हे आधीच माहीत नसताना हा योग्य पर्याय आहे — उदा. फाईल शेवटपर्यंत ओळ-ओळ वाचणे, किंवा युजरने बरोबर पर्याय निवडेपर्यंत विचारत राहणे.

Example / उदाहरण

```
#!/bin/bash
count=1

while [ $count -le 5 ]
do
    echo "Ganti kramank: $count"
    count=$((count + 1))
done

# File oLI-oLI vachnyasathi (khup common interview udaharan!)
while IFS= read -r line
do
    echo "OL: $line"
done < notes.txt

# Infinite loop (break vaparava lagto):
while true
do
    read -p "Exit sathi 'q' taka: " ans
    [ "$ans" = "q" ] && break
done
```

⚠ Common Mistakes / सामान्य चुका

- Forgetting to update the loop-control variable inside the loop (count=\$((count+1))), causing an infinite loop.
- Not using 'IFS= read -r' when reading files — this can silently trim leading/trailing whitespace or mangle backslashes.

Interview Questions & Answers

Q: How do you read a file line-by-line in bash?

A: while IFS= read -r line; do echo "\$line"; done < filename — this is a very common interview question.

Q: What's the difference between 'while' and 'until'?

A: 'while' runs as long as the condition is TRUE. 'until' runs as long as the condition is FALSE, stopping once it becomes true — they are logical opposites.

Q: How do you create an infinite loop in bash?

A: while true; do ... done (or while ;; do ... done) — combined with a 'break' statement to exit when needed.

12 until Loop

until लूप

EN 'until' is the mirror image of 'while' — instead of running while a condition is true, it keeps running as long as the condition is FALSE, and stops the instant it becomes true. It's less commonly used than while, but is handy in situations that are naturally phrased as 'keep trying UNTIL something succeeds', like waiting until a server becomes available.

MR 'until' हा 'while' च्या अगदी उलट आहे — अट खरी असताना चालण्याऐवजी, तो अट खोटी (false) असेपर्यंत चालत राहतो आणि ती खरी होताच थांबतो. 'while' पेक्षा तो कमी वापरला जातो, पण 'जोपर्यंत काहीतरी यशस्वी होत नाही तोपर्यंत प्रयत्न करत राहा' अशा परिस्थितीत उपयोगी पडतो, उदा. सर्व्हर उपलब्ध होईपर्यंत वाट पाहणे.

Example / उदाहरण

```
#!/bin/bash
count=1

until [ $count -gt 5 ]
do
    echo "Until loop: $count"
    count=$((count + 1))
done

# Practical udaharan: server ready houparyant wait karne
until ping -c1 google.com &> /dev/null
do
    echo "Server sapadat nahi, punha try karto..."
    sleep 2
done
echo "Server sapadla!"
```

⚠ Common Mistakes / सामान्य चुका

- Confusing the exit condition direction — writing 'until' logic like 'while' logic and getting the opposite behavior.

Interview Questions & Answers

Q: Give a real-world example where 'until' is more natural than 'while'.

A: Waiting for a service/server to come online: `until ping -c1 host &> /dev/null; do sleep 2; done` — it reads naturally as 'keep waiting UNTIL the ping succeeds'.

Q: Can every 'until' loop be rewritten as a 'while' loop?

A: Yes — `until [ cond ]` is logically the same as `while ! [ cond ]`, just written differently for readability.

फंक्शन्स (Functions)

EN A function is a named, reusable block of code — you write the logic once inside a function, give it a name, and then simply 'call' that name whenever you need that logic, instead of copy-pasting the same code repeatedly. Functions can accept arguments (accessed the same way as script arguments: \$1, \$2...) and can return a value using 'return' (for small integer exit-style codes) or by echoing output that gets captured by the caller. Using 'local' inside a function keeps a variable private to that function only.

MR फंक्शन म्हणजे नाव दिलेला, पुन्हा पुन्हा वापरता येणारा कोडचा भाग — एकदा त्यामध्ये लॉजिक लिहायचे, त्याला नाव द्यायचे, आणि गरज पडेल तेव्हा तेच कोड कॉपी-पेस्ट करण्याऐवजी फक्त त्या नावाने 'कॉल' करायचे. फंक्शनला आर्ग्युमेंट्स देता येतात (स्क्रिप्टच्या आर्ग्युमेंट्ससारखेच \$1, \$2... वापरून), आणि 'return' ने लहान इंटिजर कोड परत देता येतो, किंवा echo करून आउटपुट कॉलरला मिळवता येते. फंक्शनमध्ये 'local' वापरल्यास तो व्हेरिएबल फक्त त्याच फंक्शनपुरता खाजगी राहतो.

HOW IT WORKS 'return' in bash can ONLY send back a number from 0-255 (like an exit status), not text or big numbers. To get text/data back from a function, echo it and capture with result=\$(function_name).

Example / उदाहरण

```
#!/bin/bash
greet() {
    echo "Namaskar, $1! Aaj tumcha changla din javo."
}

add_numbers() {
    local sum=$(( $1 + $2 ))
    echo "$sum"          # echo का result 'return' karto
}

greet "Sneha"

result=$(add_numbers 10 20)
echo "Berij aahe: $result"

# return फक्त 0-255 sathi (success/fail dakhavnyasathi):
is_even() {
    if [ $(( $1 % 2 )) -eq 0 ]; then
        return 0      # success
    else
        return 1      # failure
    fi
}

is_even 4 && echo "Number sam (even) aahe"
```

△ Common Mistakes / सामान्य चुका

- Trying to 'return' a string or a large number directly — return only works for 0-255 exit-style codes.
- Forgetting 'local' for helper variables inside a function, accidentally overwriting a global variable of the same name.
- Calling a function before it is defined in the script (functions must be defined before they are called).

Interview Questions & Answers

Q: How do you return a text value from a bash function?

A: You can't use 'return' for text (it only accepts 0-255). Instead, echo the value inside the function and capture it in the caller using result=\$(function_name).

Q: What does 'local' do inside a function?

A: It scopes a variable to only that function, preventing it from overwriting or leaking into global variables with the same name.

Q: How do you pass arguments to a bash function?

A: The same way as script arguments — \$1, \$2, etc. refer to the values passed when the function is called, e.g. greet "Sneha" makes \$1 = "Sneha" inside greet().

अरे (Arrays)

EN A normal variable can hold only one value, but an array can hold many values under a single name — useful for lists like filenames, usernames, or numbers. Bash arrays are zero-indexed, meaning the first item is at position 0, not 1. You access a single element with `${array[index]}`, all elements with `${array[@]}`, and the total count of elements with `${#array[@]}`. Bash (version 4+) also supports associative arrays, which use string keys instead of numeric indexes, similar to a dictionary/map in other languages.

MR साध्या व्हेरिबलमध्ये फक्त एकच किंमत साठवता येते, पण अरे एका नावाखाली अनेक किंमती साठवू शकतो — फाईलनावे, युजरनेम्स, किंवा नंबरसच्या यादीसाठी उपयोगी. Bash अरे zero-indexed असतात, म्हणजे पहिला घटक 0 क्रमांकावर असतो, 1 वर नाही. एक घटक मिळवण्यासाठी `${array[index]}`, सगळे घटक मिळवण्यासाठी `${array[@]}`, आणि एकूण घटकांची संख्या मिळवण्यासाठी `${#array[@]}` वापरतात. Bash (version 4+) मध्ये associative arrays पण असतात, जे नंबर ऐवजी स्ट्रिंग की (keys) वापरतात, जसे इतर भाषांमधला dictionary/map.

Example / उदाहरण

```
#!/bin/bash
cities=("Mumbai" "Pune" "Nagpur" "Nashik")

echo "Pahile shahar: ${cities[0]}"
echo "Sagli shahare: ${cities[@]}"
echo "Ekun shahare: ${#cities[@]}"

for city in "${cities[@]}"
do
    echo "Shahar: $city"
done

cities+=("Nagar")      # nava item add karnyasathi

# Associative array (bash 4+):
declare -A capitals
capitals["Maharashtra"]="Mumbai"
capitals["Gujarat"]="Gandhinagar"
echo "Maharashtra chi rajdhani: ${capitals[Maharashtra]}"
```

⚠ Common Mistakes / सामान्य चुका

- Assuming arrays start at index 1 — bash arrays always start at 0.
- Forgetting quotes around `"${array[@]}"` in a for loop, which breaks elements that contain spaces.
- Trying to use associative arrays without 'declare -A' first, or on bash versions older than 4.

Interview Questions & Answers

Q: What index does the first element of a bash array have?

A: 0 — bash arrays are zero-indexed.

Q: How do you find the number of elements in an array?

A: `${#array_name[@]}`

Q: What is the difference between an indexed array and an associative array?

A: An indexed array uses numbers (0,1,2...) as positions. An associative array (declare -A) uses custom string keys, like a dictionary in Python — but it needs bash version 4 or above.

स्ट्रिंग हाताळणी (String Manipulation)

EN Bash has built-in ways (called 'parameter expansion') to work with strings without needing to call external programs, which makes scripts faster. You can find a string's length, extract a portion of it (substring), replace part of it, and convert it to upper/lowercase — all using special `${ }` syntax. Knowing these well is very valuable in interviews since they show you understand bash beyond just running commands.

MR Bash कडे स्ट्रिंगवर काम करण्यासाठी बिल्ट-इन पद्धती (त्याला 'parameter expansion' म्हणतात) आहेत, ज्यासाठी वेगळे प्रोग्राम बोलवावे लागत नाहीत, त्यामुळे स्क्रिप्ट जलद चालते. स्ट्रिंगची लांबी काढणे, त्यातला काही भाग (substring) काढणे, तो बदलणे, आणि upper/lowercase करणे — हे सगळे खास `${ }` सिंटॅक्स वापरून करता येते. या गोष्टी नीट माहीत असणे इंटरव्ह्यूमध्ये खूप उपयोगी ठरते, कारण त्यातून तुम्हाला bash फक्त कमांड्स चालवण्यापलीकडेही माहीत आहे हे दिसते.

Example / उदाहरण

```
#!/bin/bash
str="ShellScriptingShikuya"

echo "Lambi: ${#str}"           # string length
echo "Substring: ${str:0:5}"    # 0 pasun 5 akshar
echo "Substring shevatpasun: ${str:-6}"
echo "Badal (first match): ${str/Shell/Bash}"
echo "Badal (sagle match): ${str//S/X}"
echo "UPPERCASE: ${str^^}"
echo "lowercase: ${str,,}"

# default value dene, jar variable rikama asel tar:
unset city
echo "Shahar: ${city:-Pune}"
```

⚠ Common Mistakes / सामान्य चुका

- Confusing `${str:0:5}` (substring) with `${str/Shell/Bash}` (replace) — different syntax, different purpose.
- Not knowing `${var:-default}` for providing fallback values, which is very common in production scripts.

Interview Questions & Answers

Q: How do you get the length of a string in bash?

A: `${#string_name}`

Q: How do you replace ALL occurrences of a substring, not just the first?

A: Use double slash: `${string//old/new}` — a single slash `${string/old/new}` replaces only the first match.

Q: How would you provide a default value if a variable is unset or empty?

A: `${variable:-default_value}` — this returns 'default_value' without permanently changing the variable, while `${variable:=default_value}` also assigns it.

फाईल टेस्ट ऑपरेटर्स

EN Before a script tries to read, write, or move a file, it's good practice to first check whether that file even exists and whether the script has permission to access it — this avoids scripts crashing halfway through. Bash provides file test operators for exactly this: `-f` checks if it's a regular file, `-d` checks if it's a directory, `-e` checks if it exists at all (file or directory), and `-r`, `-w`, `-x` check read/write/execute permissions respectively.

MR स्क्रिप्ट एखादी फाईल वाचण्याआधी, तिच्यात लिहिण्याआधी, किंवा ती हलवण्याआधी ती फाईल अस्तित्वात आहे का आणि तिच्यावर परवानगी आहे का हे तपासणे चांगली सवय आहे — यामुळे स्क्रिप्ट अर्ध्यात क्रॅश होत नाही. यासाठी bash कडे file test operators आहेत: `-f` म्हणजे ती साधी फाईल आहे का, `-d` म्हणजे ती डिरेक्टरी आहे का, `-e` म्हणजे ती (फाईल किंवा डिरेक्टरी म्हणून) अस्तित्वातच आहे का, आणि `-r`, `-w`, `-x` अनुक्रमे वाचणे/लिहिणे/चालवणे यांची परवानगी तपासतात.

Example / उदाहरण

```
#!/bin/bash
file="notes.txt"

if [ -f "$file" ]; then
    echo "$file hi file astitwat aahe"
else
    echo "$file file sapadli nahi"
fi

[ -d "/home" ] && echo "/home hi directory aahe"
[ -e "$file" ] && echo "$file kontyahi swaroopat astitwat aahe"
[ -r "$file" ] && echo "$file vachta yete (readable)"
[ -w "$file" ] && echo "$file madhe lihita yete (writable)"
[ -s "$file" ] && echo "$file rikami nahi (has content)"
```

⚠ Common Mistakes / सामान्य चुका

- Confusing `-e` (exists, any type) with `-f` (exists AND is a regular file, not a directory).
- Not quoting the filename variable — `[ -f $file ]` breaks if the filename has spaces.

Interview Questions & Answers

Q: What is the difference between `-e` and `-f`?

A: `-e` checks if the path exists at all (could be a file, directory, or symlink). `-f` checks specifically that it exists AND is a regular file (not a directory).

Q: How do you check if a file is empty?

A: `! -s "$file"` is true when the file is empty (size zero). `-s "$file"` is true when it has content.

Q: How would you check if a directory exists before trying to `cd` into it?

A: `if [ -d "$dirpath" ]; then cd "$dirpath"; fi`

एक्झिट स्टेटस आणि एरर हाताळणी

EN Every single command you run in Linux, when it finishes, secretly leaves behind a number called its EXIT STATUS, stored in the special variable `$?`. A value of 0 always means success; any non-zero value (1-255) means some kind of failure, and different programs use different non-zero numbers to mean different specific errors. This is how scripts can check 'did that command actually work?' before continuing. 'set -e' is a safety option that makes the ENTIRE script stop immediately the moment any command fails, instead of blindly continuing.

MR Linux मध्ये तुम्ही चालवलेली प्रत्येक कमांड संपल्यावर, गुपचूप एक नंबर मागे सोडते ज्याला EXIT STATUS म्हणतात, तो `$ ?` या खास व्हेरिएबलमध्ये साठवला जातो. 0 म्हणजे नेहमी यशस्वी; 0 सोडून इतर कोणताही नंबर (1-255) म्हणजे काहीतरी अपयश, आणि वेगवेगळे प्रोग्राम वेगवेगळ्या विशिष्ट चुकांसाठी वेगवेगळे नॉन-झिरो नंबर वापरतात. यामुळेच स्क्रिप्ट पुढे जाण्याआधी 'ती कमांड खरंच यशस्वी झाली का ?' हे तपासू शकते. 'set -e' हा एक सुरक्षा पर्याय आहे, ज्यामुळे कोणतीही कमांड फेल होताच संपूर्ण स्क्रिप्ट लगेच थांबते, आंधळेपणाने पुढे जात नाही.

HOW IT WORKS Common convention: exit 0 = success, exit 1 = general error, exit 2 = misuse of command, 126 = permission problem, 127 = command not found, 130 = terminated by Ctrl+C.

Example / उदाहरण

```
#!/bin/bash
mkdir /tmp/test_dir
if [ $? -eq 0 ]; then
    echo "Directory yashaswuritya banli"
else
    echo "Directory banvanyat error aali"
fi

# Chukichi command aali tar script tabDavnyasathi:
set -e
cp nahiasseleli_file.txt /tmp/      # script ithech thambel

# स्वतःच्या function मधून specific exit code detene:
check_disk() {
    if [ ! -d "/backup" ]; then
        echo "Error: /backup nahi!"
        exit 1
    fi
}
```

⚠ Common Mistakes / सामान्य चुका

- Only checking `$?` after running an unrelated command in between (it always reflects the LAST command run, not any earlier one).
- Using 'set -e' but still expecting the script to continue after handling an error manually — set -e stops it unconditionally unless you specifically allow failure with '|| true'.
- Not using specific exit codes, making it hard for calling scripts/CI systems to know exactly what went wrong.

Interview Questions & Answers

Q: What does `$?` represent?

A: The exit status (return code) of the most recently executed command. 0 means success, non-zero means failure.

Q: What does 'set -e' do?

A: It tells bash to immediately stop executing the script if any command returns a non-zero (failure) exit status, instead of continuing to the next line.

Q: How do you intentionally ignore a command's failure when 'set -e' is active?

A: Append `|| true` to that command, e.g. `risky_command || true` — this always makes the line succeed even if `risky_command` failed.

Q: How do you make your own script return a specific error code?

A: Use the 'exit' command with a number, e.g. `exit 1` for a general error, or a custom code like `exit 2` for a specific known failure case.

sed आणि awk यांची तोंडओळख

EN 'sed' (Stream EDitor) and 'awk' are two extremely powerful text-processing tools that come pre-installed on almost every Linux system, and are asked about constantly in interviews. sed is mainly used to find and replace text using patterns, working line by line through input — its most common form is sed 's/old/new/'. awk is more powerful — it treats input as rows and columns (like a mini spreadsheet), letting you print, filter, or calculate based on specific columns, using \$1, \$2 for column 1, column 2, and so on.

MR 'sed' (Stream EDitor) आणि 'awk' ही दोन अतिशय ताकदवान टेक्स्ट-प्रोसेसिंग टूल्स आहेत, जी जवळपास प्रत्येक Linux सिस्टीमवर आधीच इन्स्टॉल असतात, आणि इंटरव्यूमध्ये सतत विचारली जातात. sed मुख्यतः पॅटर्न वापरून मजकूर शोधून बदलण्यासाठी वापरतात, ते इनपुट ओळ-ओळ वाचतो — त्याचा सगळ्यात सामान्य वापर म्हणजे sed 's/old/new/'. awk जास्त ताकदवान आहे — तो इनपुटला rows आणि columns (मिनी स्प्रेडशीटसारखे) समजतो, ज्यामुळे विशिष्ट कॉलमच्या आधारे print, filter किंवा गणित करता येते, \$1, \$2 वापरून column 1, column 2 असे मिळवता येतात.

Example / उदाहरण

```
#!/bin/bash
# sed - shabda badalne (pahila match)
echo "Mumbai is a big city" | sed 's/big/beautiful/'

# sed - sagle match badalne (g = global)
echo "cat bat cat" | sed 's/cat/dog/g'

# sed - oIcha number sangun delete karne
sed '2d' notes.txt          # dusri ol kadhun taknyasathi

# awk - specific column kadhne
echo "Rahul 21 Pune" | awk '{print $1, $3}'

# awk - file madhil pratyek oIcha pahila shabda
awk '{print $1}' notes.txt

# awk - ek column varun filter karne (jar age > 20)
echo -e "Rahul 21\nSneha 19" | awk '$2 > 20 {print $1}'
```

⚠ Common Mistakes / सामान्य चुका

- Forgetting the 'g' flag in sed for replacing ALL occurrences — sed 's/a/b/' only replaces the first match per line.
- Confusing sed (line/pattern based text substitution) with awk (column/field based data processing) — they solve different kinds of problems.
- Not knowing that awk automatically splits each line into fields (\$1, \$2...) by whitespace by default.

Interview Questions & Answers

Q: How do you replace ALL occurrences of a word in a line using sed, not just the first?

A: Add the 'g' (global) flag: sed 's/old/new/g'.

Q: How would you print only the 3rd column of a space-separated file using awk?

A: awk '{print \$3}' filename

Q: What is the main difference between sed and awk?

A: sed is primarily for finding/replacing text patterns line by line. awk is for processing structured, column-based data — filtering rows, printing specific fields, or doing calculations per field.

Q: How do you delete a specific line number using sed?

A: sed 'Nd' filename — for example sed '3d' file.txt deletes line 3.

19 trap & Signal Handling

trap आणि सिग्नल हाताळणी

EN A signal is a message the operating system (or a user pressing Ctrl+C) sends to a running program to interrupt it — for example SIGINT (Ctrl+C) or SIGTERM (a polite 'please stop' request). By default, a script that receives such a signal just dies immediately, possibly leaving behind temporary files or an incomplete operation. 'trap' lets your script catch a specific signal and run a chosen function first — like cleaning up temp files or printing a friendly message — before actually exiting, which is considered a mark of a well-written production script.

MR सिग्नल म्हणजे ऑपरेटिंग सिस्टीम (किंवा युजरने Ctrl+C दाबल्यावर) चालू असलेल्या प्रोग्रामला थांबवण्यासाठी पाठवलेला संदेश — उदा. SIGINT (Ctrl+C) किंवा SIGTERM (सभ्यपणे 'कृपया थांब' सांगणारा संदेश). डिफॉल्टने असा सिग्नल आल्यावर स्क्रिप्ट लगेच बंद होते, कदाचित तात्पुरत्या फाईल्स किंवा अर्धवट काम मागे ठेवून. 'trap' मुळे स्क्रिप्टला विशिष्ट सिग्नल पकडता येतो आणि प्रत्यक्ष बाहेर पडण्याआधी ठरवलेले फंक्शन चालवता येते — जसे तात्पुरत्या फाईल्स साफ करणे किंवा नीट संदेश दाखवणे — जे एक चांगल्या दर्जाच्या प्रोडक्शन स्क्रिप्टचे लक्षण मानले जाते.

Example / उदाहरण

```
#!/bin/bash
cleanup() {
    echo "Script bandh honyaadhi safai kartoy..."
    rm -f /tmp/temp_file_$$
    exit 1
}

trap cleanup SIGINT SIGTERM

echo "Script chalu aahe, Ctrl+C daabun baghaa..."
sleep 30

# EXIT var pan trap chalavta yeto (script kashahi sampali tari):
trap 'echo "Script sampli - $(date)"' EXIT
```

⚠ Common Mistakes / सामान्य चुका

- Forgetting that some signals (like SIGKILL, signal 9) cannot be trapped or ignored — they force-kill the process immediately.
- Not removing temp files created with \$\$ (process ID) inside the trap handler, defeating the purpose of using trap.

Interview Questions & Answers

Q: What is a signal in Linux?

A: A signal is a notification/message sent to a process to tell it to do something — commonly to interrupt or terminate it, like SIGINT (Ctrl+C) or SIGTERM.

Q: Can you trap the SIGKILL signal?

A: No — SIGKILL (signal 9) cannot be caught, blocked, or ignored; it forcefully terminates the process immediately, by design.

Q: Why is trap useful in production scripts?

A: It allows graceful cleanup (removing temp files, closing connections, logging) before the script actually exits, even if it was interrupted unexpectedly.

डीबगिंग आणि बेस्ट प्रॅक्टिसेस

EN When a script isn't behaving as expected, you don't have to guess — bash has a built-in debug mode. Running 'bash -x script.sh', or adding 'set -x' inside the script, prints every single command to the screen right before it runs (with variables already expanded), so you can see exactly what bash is actually doing versus what you assumed it would do. Beyond debugging, following consistent best practices — quoting variables, meaningful names, checking exit codes, adding comments — is exactly what separates a script that 'happens to work' from one that is safe to run in production and easy for someone else to maintain.

MR स्क्रिप्ट अपेक्षेप्रमाणे वागत नसेल, तेव्हा अंदाज न बांधता तपासता येते — bash कडे बिल्ट-इन डीबग मोड आहे. 'bash -x script.sh' असे रन करून, किंवा स्क्रिप्टमध्ये 'set -x' टाकून, प्रत्येक कमांड ती चालण्याआधीच स्क्रीनवर दाखवली जाते (व्हेरिएबल्सच्या खऱ्या किंमतींसह), त्यामुळे तुम्ही जे गृहीत धरले होते ते नाही तर bash खरंच काय करतोय हे दिसते. डीबगिंग व्यतिरिक्त, सातत्याने चांगल्या पद्धती पाळणे — व्हेरिएबल्स quote करणे, अर्थपूर्ण नावे देणे, exit code तपासणे, कॉमेंट्स लिहिणे — हेच 'कसंतरी चालणारी' स्क्रिप्ट आणि प्रोडक्शनमध्ये सुरक्षितपणे चालवता येणारी, दुसऱ्याला सहज समजणारी स्क्रिप्ट यातला फरक ठरवते.

Example / उदाहरण

```
#!/bin/bash
set -x      # debugging chalu

name="Amit"
echo "Namaskar $name"

set +x      # debugging bandh

# Best Practices (interview madhe vicharle jate):
# 1. "$variable" nehmi quotes madhe vapra (word-splitting tallyasathi)
# 2. Function ani variable la changli, spashta nave dya
# 3. Script chya suruvatila comment lihun kam samjava
# 4. set -euo pipefail vapra (strict mode):
#    -e : kontihi command fail zali tar thamba
#    -u : na-declared variable vaparla tar error dya
#    -o pipefail : pipe madhil konatihi command fail zali tar tya
#               entire pipeline la fail mhanava
# 5. Chuka handle karnyasathi if/exit status vapra
```

⚠ Common Mistakes / सामान्य चुका

- Forgetting to turn debugging off (set +x) after you're done, which floods production logs with every command.
- Not using 'set -euo pipefail' at the start of production scripts — a widely recommended 'strict mode' that catches many silent bugs early.
- Writing overly clever one-liners that are hard for teammates to read and maintain later.

Interview Questions & Answers

Q: How do you debug a bash script line-by-line?

A: Run it with bash -x script.sh, or add 'set -x' at the top of the script (and 'set +x' to turn it off) — this prints each command with its expanded values before executing it.

Q: What does 'set -euo pipefail' do, and why is it recommended?

A: -e stops the script on any command failure, -u errors out on use of undeclared variables, and -o pipefail makes a pipeline fail if ANY command in it fails (not just the last one). Together they catch many silent bugs that a 'default' bash script would otherwise ignore.

Q: Name three general best practices for writing production-quality shell scripts.

A: Always quote variables ("\${var}"), use 'set -euo pipefail' for strict error handling, use meaningful names and comments, check exit codes of important commands, and prefer [[]] over [] for conditionals when using bash specifically.